

ENGINE DEVELOPMENT ROADMAP

THE SEED

AAA Roadmap

Von V12-Prototype zu einer produktionsreifen
MMORPG-Engine auf Industrie-Niveau.

Alle Phasen, Arbeitspakete und technischen Spezifikationen.

Projekt: The Seed Engine | Version: V12 → AAA | Datum: Juni 2026

Schwerpunkt: Rendering · Netzwerk · ECS · Gameplay · Tools · DevOps

V13+

Inhaltsverzeichnis

1. Executive Summary

2. Ausgangsanalyse: V12 → AAA-Gap

2.1 Kritische Defizite

2.2 Architekturelle Schuld

3. Gesamtarchitektur-Zielbild

3.1 Target-Stack

3.2 Modulare Schichtenarchitektur

4. Phase 1: Fundament & Rendering (Monate 1–6)

4.1 Rendering-Engine

4.2 Low-Level Systems

4.3 Build & Platform

5. Phase 2: ECS & Parallelisierung (Monate 4–8)

5.1 Neues ECS

5.2 Memory & Allocators

5.3 Job-System

6. Phase 3: Netzwerk & Multiplayer (Monate 6–10)

6.1 UDP-Transport-Layer

6.2 Snapshot-System

6.3 Server-Architektur

7. Phase 4: Gameplay-Systeme (Monate 8–14)

7.1 KI & Pathfinding

7.2 Kampf & Loot

7.3 Soziale Systeme

7.4 Scripting

8. Phase 5: Tools & Pipeline (Monate 12–18)

8.1 Editor-Suite

8.2 Asset-Pipeline

8.3 DevOps & CI/CD

9. Phase 6: Polishing & Skalierung (Monate 16–24)

9.1 Performance & Optimierung

9.2 Plattform-Portierung

9.3 QA & Beta

10. Gesamtzeitplan & Meilensteine

11. Risiken & Mitigation

12. Anhang: Technology-Stack-Referenz

1. Executive Summary

Dieses Dokument definiert den vollstaendigen Entwicklungspfad von **The Seed V12** (aktuelles Prototype) zu einer **AAA-produktionsreifen MMORPG-Engine**. Die Roadmap umfasst sechs Hauptphasen ueber einen Zeitraum von **18 bis 24 Monaten** bei einem Kern-Team von 8-12 Entwicklern.

Die aktuelle V12-Version repraesentiert ein solides Indie-Fundament mit funktionierenden Kernsystemen:

- Server-authoritative Architektur mit Client-Prediction
- Modulare Aufteilung (core / server / client / editor)
- Data-Driven CSV-Loading fuer Quests, Skills, NPCs
- Interest Management (AOI) und Sektor-Streaming
- ImGui-basierter Editor mit Undo/Redo

Der Sprung zu AAA-Niveau erfordert jedoch fundamentale Aenderungen in nahezu jedem Subsystem. Diese Roadmap strukturiert die notwendigen Arbeiten in logisch aufeinander aufbauende Phasen, minimiert Risiken durch fruehzeitige Prototypisierung kritischer Pfade und definiert klare Abnahmekriterien pro Phase.

Target-Spezifikationen (AAA-Zustand)

Bereich	Current V12	AAA Target
Renderer	OpenGL 3.0, Toon-Shading, keine Schatten	Vulkan/D3D12, PBR, Deferred, CSM, SSAO, Bloom
Max. Entities/Sektor	~100 (linear search)	10.000+ (cache-friendly ECS)
Max. gleichzeitige Spieler	64 (TCP, single-threaded)	1.000+ pro Shard (UDP, multithreaded)
Netzwerk-Tickrate	60 Hz (TCP, unzuverlaessig)	20 Hz Snapshot + 60 Hz Input (UDP)
Plattform	Windows nur	Win/Linux/Mac + potenziell Konsolen
Content-Pipeline	Handgeschriebene CSVs, OBJ-Loader	Automatisierte Asset-Pipeline, FBX/gltf, LODs
AI	Keine (Monster stehen nur rum)	Behavior Trees, NavMesh, Hierarchical FSM
Scripting	Keines (nur C++)	Lua 5.4 + Visual Scripting im Editor
Build-System	Keines (Hand-Kompilierung)	CMake + vcpkg/conan, CI/CD

2. Ausgangsanalyse: V12 zu AAA-Gap

2.1 Kritische Defizite nach Subsystem

Rendering

Die aktuelle Render-Schicht basiert auf **OpenGL 3.0 / GLSL 1.3** — einer API, die 2008 eingefuehrt wurde und keinen Zugriff auf moderne GPU-Features bietet. Das Toon-Shader-System verwendet lediglich eine 4-stufige Diskretisierung der Diffuse-Beleuchtung. Es gibt **keine Schatten**, kein Post-Processing, keine Animation, keine Partikel und keine LODs. Die gesamte Szene wird brute-force gerendert.

Netzwerk

Der Netzwerk-Stack verwendet ausschliesslich **TCP** mit Blocking-I/O innerhalb eines Single-Threaded-Loops. Die maximum Client-Anzahl ist auf 64 hardcodiert. Es gibt keine Snapshot-Interpolation, keine Delta-Kompression und keine Bandwidth-Priorisierung. Bei 60 Hz Tickrate und voller Spielerzahl entsteht ein massiver Bandbreiten-Overhead.

ECS & Datenstrukturen

Das aktuelle "ECS" ist kein echtes Entity-Component-System. Entities sind grosse AOS-Structs mit allen Komponenten inline. Alle Lookups nutzen `O(n)` Linear-Search ueber `std::vector`. Es gibt keine Memory Pools, keine Cache-freundlichen SOA-Layouts und kein Archetype-System.

Gameplay & AI

Monster besitzen keinerlei AI-Verhalten — sie spawnen und stehen an Ort und Stelle. Es gibt kein Pathfinding, kein Aggro-System, kein Loot, keine Gilden, kein PvP und keine Instanzierung. Der gesamte Gameplay-Bereich befindet sich auf Prototype-Niveau.

Tools & Pipeline

Es gibt kein Build-System (CMake), keine Package-Manager-Integration, kein automatisiertes Asset-Import, keine Scripting-Sprache und keine CI/CD-Pipeline. Der Editor ist auf ImGui-Basics beschaenkt ohne Visual Scripting oder Terrain-Brushing in der 3D-Ansicht.

2.2 Architekturelle Schuld

Tabelle 1 Zusammenfassung der architektonischen Schuld

Kategorie	Problem	Impact	Behebungsaufwand
Globale Variablen	<code>inline</code> -Globals in <code>World.h</code> statt DI	Untestbar, nicht thread-sicher, enge Kopplung	Hoch
O(n) Lookups	<code>std::find_if</code> ueber <code>serverRegistry</code> fuer jeden Zugriff	Degradation ab ~50 Entities	Hoch
Vector-Erase	<code>erase-remove_if</code> fuer Entity-Despawn	Cache-Misses, O(n) pro Operation	Mittel
Mutex-Granularitaet	Einzelne grosse Mutexes (sessions, inventory)	Lock-Contention bei Skalierung	Mittel
Keine Allocators	<code>std::allocator default</code> fuer alle Container	Fragmentierung, kein Memory-Budgeting	Mittel
String-IDs	<code>materialId</code> , <code>meshId</code> als <code>std::string</code>	Heap-Allokationen, langsame Vergleiche	Niedrig
Argon2Id Fake	Eigene Hash-Funktion statt echter Argon2	Kryptographisch unsicher	Niedrig

3. Gesamtarchitektur-Zielbild

3.1 Target Technology-Stack

Tabelle 2 Empfohlener AAA-Technology-Stack

Layer	Technologie	Alternative	Begründung
Graphics API	Vulkan 1.3	D3D12 (Win-only)	Cross-Platform, explizite Kontrolle, Raytracing-Extension
Shader	Slang oder SPIR-V direkt	GLSL (legacy)	Shader-Varianten-Management, DXIL/SPIRV-Dual-Output
Render-Pipeline	Custom Deferred+	Forward+	PBR, viele Lichter, CSM, SSAO, Bloom
Netzwerk	ENet oder GameNetworkingSockets	Custom UDP	Zuverlässiges UDP, P2P NAT-Traversal-ready
ECS	Flecs oder Custom Archetype	EnTT, EntityX	Archetype-Speicher, System-Pipelining, C-API fuer Scripting
Physics	Jolt Physics	PhysX, Bullet	Modern, C++, deterministisch, performant
Animation	o3d-animation	Custom	Runtime-only, C++, Skeleton-Skinning, Blending
Audio	FMOD Studio	Wwise, OpenAL	AAA-Standard, 3D-Audio, Event-System
Scripting	Lua 5.4 + sol3	Wren, AngelScript	Industrie-Standard, leichtgewichtig, hot-reload
UI	dear imgui (Editor) + RmlUi (In-Game)	CoherentGT	ImGui fuer Tools, RmlUi fuer HTML/CSS-Game-UI
Build	CMake + vcpkg	Conan, xmake	Cross-Platform, Microsoft-offiziell, 900+ Ports
CI/CD	GitHub Actions	GitLab CI	Kostenlos fuer Open Source, Self-Hosted Runner
Database	PostgreSQL (Cluster)	MariaDB, ScyllaDB	ACID, Replikation, JSON-B fuer flexible Schemas
Asset-Pipeline	Custom (FBX/gltf → Engine-Format)	AssetCooker	Import, Validierung, LOD-Gen, Texture-Compression
Container/Orchestrierung	Docker + Kubernetes	Docker Swarm	Cloud-Ready, Auto-Scaling der Game-Server

3.2 Modulare Schichtenarchitektur

Abbildung 1 Engine-Schichtenarchitektur

Layer	Komponenten	Abhängigkeiten
Game Layer	Lua Scripts, Quest/Mission, AI/Behavior, Gameplay Systems	Engine Layer
Engine Layer	ECS World, Systems, Renderer, Network, Physics, Audio	Core Layer
Core Layer	Math, Memory, Platform Abstraction, IO/Serialization	Keine
Tools Layer	World Editor, Asset Pipeline, Visual Scripting, Build System	Engine Layer

4. Phase 1: Fundament & Rendering (Monate 1-6)

Team: 3-4 Renderer/DevOps | Budget: 30% der Gesamtzeit | Risiko: Hoch | Abhaengigkeiten: Keine

Ziel: Produktionsreife Render-Engine mit moderner Grafik-API

Phase 1 ist das Fundament aller nachfolgenden Arbeiten. Ohne performanten, stabilen Renderer kann kein AAA-Content dargestellt werden. Die Parallelisierung mit Low-Level-Systems (Math, Allocators, File-IO) maximiert die Effizienz.

4.1 Rendering-Engine (AP-01 bis AP-08)

Tabelle 3 Arbeitspakete: Rendering-Engine

ID	Arbeitspaket		Prio	Deliverables & Akzeptanzkriterien
AP-01	Vulkan-Renderer-Init		Kritisch	Instance, Device, Swapchain, Queue-Familien; valide Vortestung auf Win/Linux; Vulkan-Validation-Layers fehlerfrei
AP-02	Render-Pass Abstraktion	& Framebuffer-	Kritisch	Generischer RenderPass-Builder, Attachment-Description, Subpass-Dependencies; Unterstuetzung fuer Multisampling
AP-03	Resource-Manager (Descriptor Pools, Layouts)	(Descriptor	Kritisch	Automatische Descriptor-Allokation, Bindless-ready Descriptor-Set-Layout, Texture-Array-Support
AP-04	Deferred Shading Pipeline		Kritisch	G-Buffer mit 4 Render Targets (Albedo, Normal, Roughness/Metallic, Depth); MRT-RenderPass; >100 Lichter in Szenen mit 1ms Shadow-Budget
AP-05	PBR-Material-System		Hoch	Metallic-Roughness-Workflow, IBL (Diffuse + Specular Convoluted Cube Maps), Material-Instancing, Uebergang von hardcoded zu Material-Definitionen
AP-06	Cascaded Shadow Maps (CSM)		Hoch	4 Cascades mit PCF-Softening, stabile Shadow-Akne-Korrektur, Shadow-Bias basierend auf Slope, Cascade-Bleeding-Reduktion
AP-07	Post-Processing Stack		Hoch	Bloom (Mipmap-Pyramide + Blur), SSAO (GTAO oder HBAO), Tone Mapping (ACES), FXAA oder TAA, Chromatic Aberration (optional)
AP-08	Partikel-System (GPU-gestuetzt)		Mittel	Compute-Shader-basierte Simulation, Emitters (Burst, Continuous, Trail), Kollision mit Tiefenbuffer, Billboard-Rendering, 100.000+ Partikel bei 60fps

Rendering: Zusätzliche Arbeitspakete

Tabelle 4 Arbeitspakete: Rendering Ergänzungen

ID	Arbeitspaket	Prio	Deliverables
AP-09	Skeletal Mesh Rendering	Hoch	o3z-animation Integration, Vertex-Shader Skinning, Pose-Blending, Animation-Layering
AP-10	Instanced Rendering	Hoch	GPU-Instancing fuer Vegetation/Props, Draw-Call-Batching, LOD-Crossfade
AP-11	Dynamic Resolution Scaling	Mittel	Automatische Auflösungsanpassung bei FPS-Drops, upscaling via FSR 2.x oder interner TAA-Upscaler
AP-12	Debug-Render-Pass	Mittel	Wireframe-Overlay, Bounding-Box-Visualisierung, Normal-Debug-View, Light-Heatmap

4.2 Low-Level Systems (AP-13 bis AP-16)

Tabelle 5 Arbeitspakete: Low-Level Systems

ID	Arbeitspaket	Prio	Deliverables
AP-13	Math-Library (SIMD)	Kritisch	SSE4/AVX2/NEON-Vektor-Math, Matrix-Operationen, Quaternionen, Frustum-Culling, Bounding-Volumes; Header-only oder kompiliert
AP-14	Memory-Allocators	Hoch	Pool-Allocator, Stack-Allocator (linear/ring), Freelist, Slab-Allocator; Alignment-Support, Statistik/Tracking fuer Memory-Budgets
AP-15	File-IO & VFS	Hoch	Virtual-File-System (physische Dateien + gepackte Archive), asynchrones Streaming (IO-Threadpool), Memory-Mapping fuer grosse Assets
AP-16	Reflection & Serialization	Hoch	RTTR oder Custom-Reflection, Binary-Serializer fuer Chunk-basiertes Format, Versioning, Forward/Backward-Compatibility

4.3 Build & Platform (AP-17 bis AP-19)

Tabelle 6 Arbeitspakete: Build & Platform

ID	Arbeitspaket	Prio	Deliverables
AP-17	CMake-Build-System	Kritisch	CMakeLists.txt fuer alle Module, vcpkg-Manifest (vcpkg.json), Debug/Release/RelWithDebInfo-Konfigurationen, automatische Dependency-Resolution
AP-18	Platform-Abstraction-Layer	Hoch	Window-Manager (GLFW als Basis), Input-Abstraktion (Keyboard, Mouse, Gamepad), Thread-Wrapper, Timer/Profiler-Hooks
AP-19	Profiling & Instrumentation	Mittel	Integration von Tracy oder Remotery, GPU-Profiler (Vulkan-Query-Pools), automatische Frame-Time-Statistiken, Memory-Tracking-Overlay

Phase 1 Risiken

R1.1 — Vulkan-Lernkurve: Vulkan erfordert 2000+ LOC nur fuer Initialisierung. Mitigation: Verwendung von `vk-bootstrap` als Hilfsbibliothek fuer Boilerplate-Reduktion.

R1.2 — Scope-Creep im Renderer: Tendenz, zu frueh Raytracing oder Mesh-Shading zu implementieren. Mitigation: Strict Deferred-Only fuer Phase 1, RT als Phase-6-Erweiterung.

5. Phase 2: ECS & Parallelisierung (Monate 4–8)

Team: 2-3 Engine-Core | Ueberlappung mit Phase 1 (Monate 4-6) | Risiko: Mittel-Hoch

Ziel: Cache-freundliches ECS mit Job-Parallelisierung

Das aktuelle V12-Pseud-ECS muss komplett ersetzt werden. Ein Archetype-basiertes ECS ist Voraussetzung fuer Skalierung auf 10.000+ Entities pro Sektor und fuer effiziente System-Parallelisierung.

5.1 Neues ECS-Kernsystem (AP-20 bis AP-24)

Tabelle 7 Arbeitspakete: ECS-Kernsystem

ID	Arbeitspaket	Prio	Deliverables
AP-20	Archetype Storage Design	Kritisch	SOA-Chunk-basierte Speicherung (16KB Chunks), Entity-IDs als generational handles (32-bit index + 32-bit generation), Archetype-Hash aus Component-Bitset
AP-21	Entity-Manager & Queries	Kritisch	Create/Destroy/Add/RemoveComponent, Query-API mit All/Any/None-Filtern, Query-Caching, O(1) Entity-Lookup ueber Sparse-Set
AP-22	System-Registry & Scheduling	Kritisch	System-Registration mit Phase/Group-Tag, automatisches Dependency-Graph-Builden aus Komponenten-Lese/Schreib-Deklarationen, Topo-Sort fuer Ausfuehrungsreihenfolge
AP-23	World-Migration von V12	Hoch	Adapter-Layer der V12-Datenstrukturen (Server-Registry, Inventare, Quest-Logs) in ECS-Component-Form abbildet; Migrationstool fuer bestehende Savesgames
AP-24	ECS-Unit-Tests & Benchmarks	Hoch	GoogleTest-Suite fuer alle ECS-Operationen, Cache-Miss-Benchmark (vs. V12-Vector), Memory-Usage-Vergleich, 10k/50k/100k-Entity-Stresstests

5.2 Memory-Management & Allocators (AP-25 bis AP-27)

Tabelle 8 Arbeitspakete: Memory-Management

ID	Arbeitspaket	Prio	Deliverables
AP-25	ECS-Chunk-Allocator	Hoch	Pool-Allokation fuer 16KB-Chunks, automatische Chunk-Verwaltung (alloc/free/reuse), Alignment auf 64-Byte-Cacheline
AP-26	Component-Serializer	Hoch	Automatische (de)serialisierung aller Component-Typen via Reflection, Chunk-basiertes Save/Load, Delta-Serialization fuer Netzwerk
AP-27	String-Interning & Hash-IDs	Mittel	StringHash32/64 als Ersatz fuer std::string in Hotpaths, String-Interning-Pool fuer Material-Namen, Mesh-IDs, Tags

5.3 Job-System & Parallelisierung (AP-28 bis AP-31)

Tabelle 9 Arbeitspakete: Job-System

ID	Arbeitspaket	Prio	Deliverables
AP-28	Fibers/Job-Queue-Implementierung	Kritisch	Work-Stealing-Deque pro Worker-Thread, Fiber-basierte Job-Ausfuehrung (oder thread-pool mit Kontinuationen), Lock-freie Task-Submission
AP-29	Parallel-For & Reduction	Hoch	ParallelFor ueber Component-Arrays, ParallelReduction fuer Aggregationsoperationen (z.B. Spatial-Hash-Built), Grain-Size-Auto-Tuning
AP-30	Sync-Primitive (lightweight)	Hoch	Fiber-Semaphore, Countdown-Latch, Future/Promise fuer Job-Dependencies, kein std::mutex in Hotpaths
AP-31	System-Parallelisierung	Mittel	Automatische System-Ausfuehrung basierend auf Dependency-Graph, Burst-Compiler-aenliche SIMD-Iteration ueber Component-Arrays

Phase 2 Risiken

R2.1 — ECS-API-Breaking-Changes: Alle bestehenden Systeme müssen migriert werden.
Mitigation: Schrittweise Migration — erst ECS-Core, dann ein Subsystem nach dem anderen.

R2.2 — Fiber-Portabilität: Fibers sind platform-spezifisch (Windows: SwitchToFiber, Linux: makecontext). Mitigation: Abstraktionslayer oder Fallback auf Thread-Pool mit Work-Stealing.

6. Phase 3: Netzwerk & Multiplayer (Monate 6–10)

Team: 2-3 Netzwerk/Backend | Ueberlappung mit Phase 2 | Risiko: Kritisch

Ziel: UDP-basierter, skalierbarer Netzwerk-Stack fuer 1.000+ CCU

Das Netzwerk ist das Herz einer MMORPG-Engine. Der Wechsel von TCP zu UDP mit zuverlaessigem Reliability-Layer, Snapshot-Interpolation und Multithreading ist der schwierigste technische Sprung in der gesamten Roadmap.

6.1 UDP-Transport-Layer (AP-32 bis AP-36)

Tabelle 10 Arbeitspakete: UDP-Transport

ID	Arbeitspaket	Prio	Deliverables
AP-32	UDP-Socket-Layer	Kritisch	Non-blocking UDP-Sockets (Win/Linux), Socket-Reuse fuer Multiple-Server-Prozesse, IPv4/IPv6-Dual-Stack, Send/Recv mit Scatter-Gather
AP-33	Zuverlaessiges UDP (Reliability)	Kritisch	Sequenznummern, ACK-Bitmap (Selective ACK), Retransmission-Timer mit RTT-Schaetzung (Jacobson/Karn), Fragmentierung fuer grosse Pakete (>MTU)
AP-34	Connection-Management	Kritisch	3-Wege-Handshake, Heartbeat/Keepalive, Graceful Disconnect, Timeout-Detection, Connection-Limit pro IP (DDoS-Protection)
AP-35	Packet-Priorisierung & Bandwidth	Hoch	Mehrere Send-Queues (Critical/Reliable/Unreliable), Bandwidth-Throttling pro Client, Paket-Priorisierung basierend auf Distanz/Relevanz
AP-36	Crypto-Layer	Hoch	DTLS-aenliche Verschluesselung (ChaCha20-Poly1305 oder AES-GCM), Key-Derivation bei Handshake, Replay-Protection mit Sliding-Window

6.2 Snapshot-System (AP-37 bis AP-41)

Tabelle 11 Arbeitspakete: Snapshot-System

ID	Arbeitspaket	Prio	Deliverables
AP-37	World-Snapshot-Builder	Kritisch	Serialisierung des gesamten ECS-World-States in einen Snapshot-Buffer, Delta-Kompression gegenueber vorherigem ACK'd Snapshot, Entity-Priority basierend auf Distanz/Interest
AP-38	Client-Side Interpolation	Kritisch	Snapshot-Buffer (min. 3 Snaps), Entity-Interpolation zwischen Snap N und N+1, Extrapolation bei Packet-Loss (max. 200ms), lokale Entity-Prediction
AP-39	Entity-Priority & LOD-Netzwerk	Hoch	Distanz-basierte Update-Rate (nahe: 20Hz, fern: 5Hz), Culling von irrelevanten Properties, Aggregation von kleinen Updates
AP-40	Interest-Management (Rewamp)	Hoch	Spatial-Hashing oder Quadtree fuer schnelle AOI-Queries, Layer-basiertes Interest (PvP-Zone, Dungeon-Instance), Subscription-basiert statt Broadcast
AP-41	Network-Profiler & Debug	Mittel	Bandbreiten-Monitor pro Client, Packet-Loss-Statistiken, RTT-Histogramm, Network-Graph (sent/received/loss ueber Zeit)

6.3 Server-Architektur & Skalierung (AP-42 bis AP-46)

Tabelle 12 Arbeitspakete: Server-Architektur

ID	Arbeitspaket	Prio	Deliverables
AP-42	Multi-Threaded Server	Kritisch	Netzwerk-Thread (IOCP/epoll), Simulations-Thread(s), DB-Worker-Thread, Lock-freie Message-Queues zwischen Threads
AP-43	Zone-Server-Architektur	Hoch	Mehrere Server-Prozesse fuer unterschiedliche Zonen, Seamless-Handoff zwischen Zonen, Cross-Server-Entity-Referenzen
AP-44	Instancing-System	Hoch	Dynamische Dungeon-/Raid-Instanzen, Instanz-Templates aus Data-Files, automatische Server-Allokation fuer Instanzen
AP-45	Authentication-Service	Hoch	JWT-basierte Auth, echte Argon2id (libsodium), Account-DB (PostgreSQL), Rate-Limiting fuer Login-Versuche
AP-46	Game-Datenbank (Cluster)	Mittel	Migration von SQLite zu PostgreSQL, Connection-Pooling, automatische Replikation, Caching-Layer (Redis) fuer hot data

Phase 3 Risiken

R3.1 — UDP-Firewall-NAT: UDP kann von Firewalls blockiert werden. Mitigation: Fallback-Mechanismus auf TCP fuer verbindungsproblematische Clients, NAT-Punchthrough fuer P2P-Szenarien.

R3.2 — Snapshot-Bandbreite: Bei 1.000 Entities und 20Hz kann die Bandbreite pro Client >100KB/s werden. Mitigation: Aggressive Delta-Kompression, Distanz-basierte Update-Rate-Reduktion, Property-Level-Culling.

7. Phase 4: Gameplay-Systeme (Monate 8–14)

Team: 3-4 Gameplay/AI | Ueberlappung mit Phase 3 | Risiko: Mittel

Ziel: Vollstaendige AAA-Gameplay-Feature-Suite

Die Gameplay-Phase transformiert den technischen Prototypen in ein spielbares MMORPG. AI, Pathfinding, Kampf-Systeme und soziale Features sind die Unterscheidungsmerkmale zwischen Tech-Demo und Spiel.

7.1 KI & Pathfinding (AP-47 bis AP-52)

Tabelle 13 Arbeitspakete: KI & Pathfinding

ID	Arbeitspaket	Prio	Deliverables
AP-47	NavMesh-Generation	Kritisch	Recast-Integration, automatische NavMesh-Baked aus Terrain-Daten, Tile-basierte NavMesh fuer grosse Welten, Off-Mesh-Links (Sprungpunkte, Tueren)
AP-48	Pathfinding (Crowd)	Kritisch	Detour-Crowd fuer Pfadfindung + Kollisionsvermeidung, Pfad-Smoothing, async. Pfadberechnung (Job-System), Pfad-Invalidierung bei Terrain-Aenderungen
AP-49	Behavior Tree Runtime	Kritisch	BT-Runtime (Custom oder using behaviortree.cpp), Node-Typen: Sequence, Selector, Parallel, Decorator (Loop, Invert, Timeout), Blackboard fuer Shared Memory
AP-50	AI-Behavior-Library	Hoch	Patrol, Guard, Chase, Flee, Attack, Flee-to-Spawn, Return-Home, Social (NPC-Taegliche-Routinen), BT-Templates pro Creature-Type
AP-51	Aggro & Threat-System	Hoch	Threat-Table pro Monster, Aggro-Radius + Sichtlinie, Taunt/De-Taunt, Aggro-Decay, Leash-Mechanismus (Return bei zu grossem Abstand)
AP-52	Group-AI (Formation)	Mittel	Leader-Follower-Verhalten, Formation-Patterns (Line, Wedge, Circle), Group-Coordination fuer Elite-Mobs

7.2 Kampf, Loot & Economy (AP-53 bis AP-58)

Tabelle 14 Arbeitspakete: Kampf & Economy

ID	Arbeitspaket	Prio	Deliverables
AP-53	Combat-System (Erweitert)	Kritisch	Hitbox-System (Box/Sphere/Capsule), Combo-System, Damage-Calulation (Armor, Resistance, Critical), Directional-Blocking/Parry, Death/Respawn-Flow
AP-54	Loot-System	Hoch	Loot-Tables mit Gewichtung, Conditional-Drops (erste Kill, Quest-Item), Drop-Visualisierung (Partikel auf Boden), Loot-Rollen (Need/Greed/Pass fuer Gruppen)
AP-55	Progression-System	Hoch	XP-Berechnung mit Diminishing>Returns, Level-Up mit Stat-Points, Skill-Trees (Data-Driven), Prestige/Ascension-System
AP-56	Crafting-System	Mittel	Rezepte mit Ingredients, Crafting-Stationen, Skill-basierte Erfolgschance, Quality-Tiers (Common bis Legendary)
AP-57	Wirtschaft & Auction House	Mittel	Spieler-getriebene Wirtschaft, NPC-Haendler mit Preis-Anpassung, Auction House (Bieten, Sofortkauf), Waehrungssystem (Gold + Premium)
AP-58	Mounts & Transport	Niedrig	Mount-System mit verschiedenen Tier-Typen, Speed-Bonus, Flug-Mounts (ab bestimmtem Level), Oeffentliche Transporte (Schiffe, Zeppeline)

7.3 Soziale Systeme (AP-59 bis AP-63)

Tabelle 15 Arbeitspakete: Soziale Systeme

ID	Arbeitspaket	Prio	Deliverables
AP-59	Gilden-System	Hoch	Grundung/beitritt/austritt, Rollen (Leader/Officer/Member), Gilden-Chat, Gilden-Bank, Gilden-Level/Perks, Gilden-Haus (Instanz)
AP-60	PvP-System	Hoch	Duell-Einladung, Open-World-PvP-Zonen, Battlegrounds (Instanziert), Honor-System, Ranking (ELO-basiert)
AP-61	Gruppen-System	Hoch	Party-Einladung (bis 5 Spieler), Raid-Gruppen (bis 40), Loot-Master-Modi, Rollen-Icons, automatisches Level-Scaling
AP-62	Friends & Block	Mittel	Freundesliste (Online/Offline/In-Instance), Anwesenheits-Status, Block-Liste, Freundschafts-Einladung mit Bestaetigung
AP-63	Chat-System (Erweitert)	Mittel	Kanaele (Global, Zone, Guild, Party, Raid, Whisper), Chat-Ban/Timeout, Link-Vorschau fuer Items, Chat-Filter

7.4 Scripting-Integration (AP-64 bis AP-67)

Tabelle 16 Arbeitspakete: Scripting

ID	Arbeitspaket	Prio	Deliverables
AP-64	Lua-Runtime-Integration	Kritisch	Lua 5.4 + sol3 Binding, ECS-Component-Zugriff aus Lua, Event-System (Lua kann auf Engine-Events registrieren), Sandboxing (kein File-IO aus Lua)
AP-65	Scriptable-Quest-System	Hoch	Quests voellständig in Lua definierbar, Conditions/Triggers/Actions in Script, Quest-State-Machine, Unterstuetzung fuer verzweigte Quests
AP-66	Scriptable-AI-Behaviors	Hoch	BT-Leaf-Nodes in Lua schreibbar, Custom-AI-Behavior pro NPC-Typ via Script, Hot-Reload fuer AI-Scripts
AP-67	Visual-Scripting-Editor	Mittel	Node-Graph fuer Quest-Fluesse (ImGui), Blueprint-to-Lua-Code-Generation, Live-Debugging (Breakpoint, Step-Through)

Phase 4 Risiken

R4.1 — Lua-Performance: Lua kann bei vielen Concurrent-Entities zum Bottleneck werden. Mitigation: LuaJIT (wenn verfuegbar) oder Heavy-Lifting in C++-Systemen, Lua nur fuer Glue-Code.

R4.2 — Feature-Creep: MMORPG-Features haben unendlichen Scope. Mitigation: Strikte MVP-Definition pro Feature, erst Core-Loop stabilisieren, dann Erweiterungen.

8. Phase 5: Tools & Pipeline (Monate 12–18)

Team: 2-3 Tools/Editor | Ueberlappung mit Phase 4 | Risiko: Mittel

Ziel: Professionelle Toolchain fuer Content-Creator

Eine Engine lebt von ihren Tools. Diese Phase verwandelt den rudimentaeren ImGui-Editor in eine vollwertige Content-Creation-Suite mit visuellem Scripting, automatischer Asset-Pipeline und integriertem Debugging.

8.1 Editor-Suite (AP-68 bis AP-74)

Tabelle 17 Arbeitspakete: Editor-Suite

ID	Arbeitspaket	Prio	Deliverables
AP-68	Editor-Windowing (Docking)	Hoch	ImGui-Docking-Branch mit persistenter Layout-Speicherung, multi-monitor Support, detachable Panels, Projekt-basierte Konfiguration
AP-69	3D-Viewport mit Gizmos	Hoch	ImGuizmo-Integration (Translate/Rotate/Scale), Grid-Snapping, Multi-Select, Viewport-Camera (Fly/Orbit), Scene-Picking (Raycast)
AP-70	Entity-Outliner & Inspector	Hoch	Hierarchische Entity-Ansicht (drag-and-drop parenting), Component-Inspector (Reflection-generierte UI), Component Add/Remove/Edit
AP-71	Terrain-Editor (Erweitert)	Mittel	3D-Paint-Brushing (Hoehe/Textur/vegetation), Echtzeit-Preview, Terrain-Layer-System (Grass, Rock, Snow), automatische LOD-Generation
AP-72	Visual-Scripting-Editor	Mittel	imnodes-Integration, Quest-Graph, AI-Behavior-Graph, Event-Node-System, Real-Time-Validation, Lua-Code-Export
AP-73	Animation-Editor (Basis)	Mittel	Animation-State-Machine-Editor, Blend-Tree-Visualisierung, Animation-Preview im Viewport
AP-74	Database-Editor	Niedrig	Tabellarischer Editor fuer Item/Quest/NPC/Skill-Daten, Import/Export CSV/JSON, Referenz-Validierung

8.2 Asset-Pipeline (AP-75 bis AP-79)

Tabelle 18 Arbeitspakete: Asset-Pipeline

ID	Arbeitspaket	Prio	Deliverables
AP-75	Asset-Importer	Kritisch	FBX-Import (assimp), glTF 2.0 (Import + als Standard-Format), automatisches Mesh-Validierung, Skeleton-Extract
AP-76	Asset-Cooking	Hoch	Plattform-abhaengige Kompilierung (Win/Linux texture formats), automatische LOD-Generation (meshoptimizer), Meshlet-Generation fuer GPU-Driven-Pipeline
AP-77	Texture-Pipeline	Hoch	BC7/ASTC-Kompression (bc7e/astc-encoder), Mipmap-Generierung, Texture-Atlas-Building, Normal-Map-Validierung
AP-78	Asset-Browser	Mittel	ImGui-basierter Datei-Browser, Thumbnail-Preview, Drag-and-Drop in Viewport, Folder-basierte Organisation, Search/Filter
AP-79	Hot-Reload-System	Mittel	File-Watcher fuer Asset-Aenderungen, automatisches Reimport bei Aenderung, Live-Reload von Shaders, Texturen, Lua-Scripts

8.3 DevOps & CI/CD (AP-80 bis AP-84)

Tabelle 19 Arbeitspakete: DevOps & CI/CD

ID	Arbeitspaket	Prio	Deliverables
AP-80	GitHub-Actions-Pipeline	Hoch	Build-Matrix (Win/Linux, Debug/Release), automatische Unit-Test-Ausfuehrung, Code-Coverage (gcov + codecov), Artifact-Speicherung
AP-81	Container-Deployment	Hoch	Dockerfiles fuer Game-Server, Auth-Service, DB, Docker-Compose fuer lokale Entwicklung, Kubernetes-Manifests fuer Production
AP-82	Monitoring & Alerting	Mittel	Prometheus + Grafana fuer Server-Metriken (CCU, FPS, Latenz, Memory), automatische Alerts bei Anomalien, ELK-Stack fuer zentrales Logging
AP-83	Deployment-Automatisierung	Mittel	Blue/Green-Deployment fuer Zero-Downtime, automatische DB-Migrationen, Rollback-Mechanismus, Feature-Flags fuer A/B-Testing
AP-84	Crash-Reporter & Analytics	Niedrig	Automatischer Crash-Report (minidump + Stacktrace), Breakpad-Integration, Analytics-Pipeline (Spieler-Verhalten, Retention), GDPR-konforme Datensammlung

9. Phase 6: Polishing & Skalierung (Monate 16–24)

Team: 3-4 Mixed (Performance, QA, Portierung) | Risiko: Mittel

Ziel: Produktionsreife, Performance-Optimierung und Plattform-Ausbreitung

Die finale Phase konzentriert sich auf das Polieren aller Subsysteme, Performance-Optimierung fuer Massenskalierung und die Portierung auf zusätzliche Plattformen. Dies ist die Phase der "letzten 20%" die 80% der Wahrnehmung ausmachen.

9.1 Performance & Optimierung (AP-85 bis AP-89)

Tabelle 20 Arbeitspakete: Performance

ID	Arbeitspaket	Prio	Deliverables
AP-85	GPU-Driven-Rendering	Hoch	Mesh-Shader oder Compute-basiertes Culling, Indirect-Draw-Batches, Automatische LOD-Selektion auf GPU, 10x Draw-Call-Reduktion
AP-86	Occlusion Culling	Hoch	Software-Occlusion-Culling (Intel-OLDN-basiert) oder Hardware-Z-Buffer-Occlusion, Portal-System fuer Innenregionen, automatisches Occluder-Generieren
AP-87	Streaming & Loading	Hoch	World-Streaming (Sektoren asynchron laden), Predictive-Loading basierend auf Bewegungsrichtung, Background-Asset-Decompression, Loading-Screen-freie Uebergaenge
AP-88	Server-Performance	Hoch	Entity-Update-Batching, Spatial-Hash fuer AOI (O(1) Query), lazy Component-Updates, Server-Side LOD (nahe: volle Sim, fern: vereinfacht)
AP-89	Memory-Optimierung	Mittel	Memory-Budgets pro Subsystem, automatische Texture-Downgrade bei Druck, Asset-Unloading bei Nichtverwendung, Memory-Defragmentierung

9.2 Plattform-Portierung (AP-90 bis AP-93)

Tabelle 21 Arbeitspakete: Portierung

ID	Arbeitspaket	Prio	Deliverables
AP-90	Linux-Client (Vulkan)	Hoch	Volle Linux-Unterstützung, Steam-Deck-Kompatibilität, Wayland + X11, Controller-Support (SDL2 Gamepad)
AP-91	macOS-Port (MoltenVK)	Mittel	MoltenVK-Integration für macOS/iOS, Metal-Performance-Shader-Fallback, Apple-Silicon-Optimierung (ARM64)
AP-92	Konsolen-Vorbereitung	Mittel	Abstraktion für proprietäre Graphics-APIs, Memory-Layout für konsolenspezifische Einschränkungen, Certification-Requirements-Checkliste
AP-93	Controller & Accessibility	Mittel	Volle Controller-Unterstützung (Xbox/PS), Tastatur-Remapping, UI-Skalierung, Farbenblinden-Modi, Untertitel-System

9.3 QA, Beta & Balancing (AP-94 bis AP-97)

Tabelle 22 Arbeitspakete: QA & Beta

ID	Arbeitspaket	Prio	Deliverables
AP-94	Automated-Testing-Suite	Hoch	Unit-Tests (>80% Coverage), Integration-Tests für Netzwerk, Headless-Server-Tests, Performance-Regression-Tests
AP-95	Load-Testing	Hoch	Bot-Simulation für 1000+ gleichzeitige Clients, Server-Stresstest, Netzwerk-Latenz-Simulation, Lastspitzen-Tests
AP-96	Closed-Beta & Feedback	Mittel	100-500 Beta-Tester, Bug-Tracking-System, Feedback-Portal, Balancing-Daten aus Analytics
AP-97	Polishing-Pass	Mittel	Visuelles Polishing (Effekte, Animation), Audio-Finalisierung (FMOD-Mixing), Performance-Finalisierung, Lokalisierung (DE/EN/FR/JP)

10. Gesamtzeitplan & Meilensteine

Phasen-Uebersicht

Tabelle 23 Phasen-Zeitplan

Phase	Zeitraum	Dauer	Team	Meilenstein
P1: Rendering Fundament	M1 – M6	6 Monate	3-4	Erste Szene mit PBR + Schatten + PostFX
P2: ECS & Parallelisierung	M4 – M8	5 Monate	2-3	10k Entities @ 60fps, ECS-Unit-Tests gruen
P3: Netzwerk Multiplayer	M6 – M10	5 Monate	2-3	100 CCU-Stresstest bestanden, Latenz <100ms
P4: Gameplay-Systeme	M8 – M14	7 Monate	3-4	Spielbarer Loop: Quest → Kampf → Loot → Level
P5: Tools & Pipeline	M12 – M18	7 Monate	2-3	Externer Content-Creator kann Inhalte erstellen
P6: Polishing & Skalierung	M16 – M24	9 Monate	3-4	Closed Beta, Linux-Port, 1000 CCU

Hinweis: Phasen ueberlappen bewusst (z.B. beginnt ECS-Design in M4 waehrend Rendering laeuft). Die Gesamtdauer betraegt 24 Monate mit sequentiellen Meilensteinen oder 18 Monate bei voller Parallelitaet und ausreichend Team-Groesse.

Kritische Pfad-Analyse

Abbildung 2 Phasen-Abhaengigkeiten (kritischer Pfad fett)

Phase	Zeitraum	Benoetigt	Parallel moeglich
P1: Rendering	M1-M6	—	—
P2: ECS	M4-M8	P1 (Renderer-Init)	P1
P3: Netzwerk	M6-M10	P1, P2	P2
P4: Gameplay	M8-M14	P1, P2, P3	P3
P5: Tools	M12-M18	P4 (Gameplay-APIs)	P4
P6: Polish	M16-M24	P4, P5	P5

Kritischer Pfad: P1 (Renderer) → P2 (ECS) → P3 (Network) → P4 (Gameplay) → P6 (Polish) = **20 Monate**. P5 (Tools) kann parallel zu P4 laufen. Verzoegerungen in P1 oder P2 wirken sich direkt auf alle nachfolgenden Phasen aus.

11. Risiken & Mitigation

Tabelle 24 Risiko-Matrix

Risiko		Wahrscheinlichkeit	Impact	Mitigation
R-VL-01:	Vulkan-	Mittel	Kritisch	vk-bootstrap fuer Boilerplate; Spike-Prototype vor Commit; Fallback-Plan mit BGFX
Lernkurve zu steil				
R-EC-02:	ECS-Migration	Hoch	Hoch	Schrittweise Migration; Adapter-Pattern; erst Core dann Subsysteme
laeuft aus dem Ruder				
R-NT-03:	UDP-Netzwerk	Mittel	Kritisch	ENet als solide Basis statt from-scratch; fruehe Stresstests; TCP-Fallback
instabil				
R-SC-04:	Scope-Creep	Sehr hoch	Mittel	Strikte MVP-Definition; Feature-Flags; erst Core-Loop dann Erweiterungen
bei Gameplay				
R-PE-05:	Performance	Mittel	Hoch	Fruehe Performance-Budgets; regelmaessige Profiling-Sessions; automatische Regression-Tests
nicht erreichbar				
R-TE-06:	Team nicht	Mittel	Hoch	Modulare Architektur ermoeoglicht parallele skalierbar
skalierbar				
R-DP-07:	Dependencies	Niedrig	Mittel	vcpkg-Pinning auf Versionen; Forking kritischer Libs; Abstraktionsschichten
brechen				
R-SE-08:	Security-	Mittel	Kritisch	Server-authoritative Design beibehalten; Crypto-Layer; Anti-Cheat-Grundlagen; Rate-Limiting
Verletzungen				

12. Anhang: Technology-Stack-Referenz

Empfohlene Bibliotheken pro Subsystem

Tabelle 25 Bibliotheks-Referenz

Subsystem	Erste Wahl	Alternative	Lizenz
Vulkan-Bootstrap	vk-bootstrap	volk	MIT
Window/Input	GLFW	SDL2	Zlib
Math	glm	DirectXMath	MIT
ECS	Flecs	EnTT	MIT
Physics	Jolt Physics	PhysX 5	MIT
Animation	ozz-animation	Custom	MIT
Audio	FMOD	Wwise, miniaudio	Proprietaer / MIT
Netzwerk	GameNetworkingSockets	ENet, yojimbo	BSD / MIT
Scripting	Lua 5.4 + sol3	Wren, AngelScript	MIT
UI (In-Game)	RmlUi	CoherentGT	MIT / Proprietaer
UI (Editor)	Dear ImGui	—	MIT
Reflection	RTTR	Custom	MIT
Mesh-Processing	meshoptimizer	—	MIT
NavMesh	Recast + Detour	—	Zlib
Behavior Trees	BehaviorTree.CPP	Custom	MIT
Asset-Import	assimp	tinyglTF	BSD
Texture-Compression	bc7e, astc-encoder	Compressorator	MIT / Apache
Serialization	flatbuffers	protobuf, MessagePack	Apache
Hashing/Security	libsodium	OpenSSL	ISC
Testing	GoogleTest	Catch2	BSD
Profiling	Tracy	Remotery	BSD
Build	CMake + vcpkg	xmake + conan	BSD / MIT
Database	PostgreSQL + libpq	MariaDB	PostgreSQL
Container	Docker	podman	Apache

Geschätzter Budget-Impact der Bibliotheken

Tabelle 26 Kostenuebersicht

Bibliothek	Kosten	Hinweis
FMOD Studio	5.000 \$ / Titel	Indie-Lizenz verfügbar; bei Umsatz >200k\$ erforderlich
Wwise	Kostenlos bis 150k\$ Umsatz	Alternative zu FMOD; danach pro-Titel-Lizenz
Alle anderen Bibliotheken	Kostenlos (Open Source)	MIT/BSD/Apache/Zlib — kommerziell nutzbar

Zusammenfassung & Naechste Schritte

Die gesamte Roadmap umfasst **97 Arbeitspakete** in **6 Phasen** ueber **18-24 Monate**. Der kritische Pfad fuehrt ueber Rendering → ECS → Netzwerk → Gameplay. Fuer ein Solo-Projekt empfiehlt sich ein fokussierter Ansatz: Phase 1+2 als erstes Ziel (ein funktionierender Renderer mit modernem ECS), dann inkrementelle Erweiterungen.

Unmittelbare Empfehlung: Mit AP-01 (Vulkan-Init) und AP-17 (CMake) beginnen, da diese keinerlei Abhaengigkeiten haben und das Fundament fuer alle nachfolgenden Arbeiten bilden.